

Why CFF generation slows down

Expression structure · Tensor cost · Causal evidence

2026-09-15 · Source-pinned investigation

The main C3 regression starts before tensor execution. Complete UV expressions lose common factors and replace shared, sign-parametric numerators with independently mapped branch copies. The tensor evaluator consequently sees larger tensor-valued sums and smaller scalar-only subexpressions. Fixed-executor experiments prove that this changed structure materially increases contraction time and memory. The separate C6 integrated delay is a nonterminating relative epsilon-series search on an algebraically zero coefficient. C7/C8 also expose a color-trace parser defect before integration.

The [native measurement matrix](#) owns the full-card timings, exact source/dependency pins and workload definitions. C1/C2 GL1 generation is about 39 seconds versus 166 seconds at C3. The attachment's common sparse order passes at C1/C2 and exceeds 30 GB from C3. The two sunrise workloads remain inexpensive. These comparisons use each revision's native locked dependencies; C1–C8 share the same Symbolica fork revision. All interventions below preserve the production workspace and published branch.

Expression structure, not just expression length

A tensor-valued sum must have the same exposed indices in every arm. If an arm contains a shared vector or metric, leaving that factor inside the sum makes its indices part of the tensor being materialized. Rank is therefore more informative than source-text length: a rank-six Lorentz tensor has 4^6 possible components, while a rank-three tensor has 4^3 . Each component may itself contain a large symbolic expression.

The relevant distinction is between $T * (a + b)$ and $T*a + T*b$, including nested forms. The latter can repeat both tensor work and the scalar expressions broadcast into its components. Products of sums may remain syntactically unexpanded while still losing enough sharing to become expensive. A strategy can choose a better contraction order for that input; it cannot recover a shared numerator that was already copied into separate upstream branches.

C1 calls `collect_factors().simplify_metrics()` after multiplying the cograph numerator in `uv/approx/final_integrand.rs`. C3's `simplify_final` omits collection to retain natural Taylor denominator sums. That preserves an important denominator contract but also drops common tensor-factor recovery. The native collector cannot simply be restored globally: it can turn $G*A/D1 + G*B/D2$ into $G*(A*D2+B*D1)/(D1*D2)$. The controlled extractions below preserve complete denominator factors and never distribute a graph numerator.

A second boundary is energy mapping. C1 reaches the vector compactor before the aggregate time/spatial split; C3 maps a vector to $Q3 + E*\delta$ first. The unchanged compactor then sees an additive vector rather than the rank-one function it can compact. The complete bare attachment witness shows compact gamma arguments becoming explicit indexed products, with the momentum sums still factorized. This is a separate witness: neither version of the capped UV owner below contains compact gamma arguments, and the hot GL1 contributions are gamma-free. Their demonstrated cause is lost sharing and larger tensor sums.

Matched attachment contribution: shared metrics

The complete contribution belongs to source 16C4 and forest {16C4} on both C1 and C3. The C3 input has the form

```
Gamma_product * [g(10,11)*g(12,13)*A + g(10,11)*g(12,13)*B]
```

where the twelve gamma factors are already outside the sum. Moving the two identical metrics outside gives

```
Gamma_product * g(10,11)*g(12,13) * [A+B].
```

The inner sum's exposed Lorentz rank falls $10 \rightarrow 6$; its possible component space falls by 256. This is a component-space bound, not a measured allocation ratio. The matching C1 owner already has the rank-six organization. The complete original and factored C3 inputs are exactly equal, including coefficients, signs, selectors, indices and denominators, as certified by factor multisets and reversible replacement. Complete C1/C3 scalar-coefficient equality is not asserted.

One immutable C3 executor is used for both inputs, with the same aliases and completion pass:

Complete contribution	Original	Shared metrics outside
Parse/prepare/contract/restore interval	11.300339s	0.738415s
Process-tree peak RSS	6.1518 GB	0.4805 GB
Packed scalar Atom	56,239,868 bytes	22,430,444 bytes
Guarded process wall	17.4167s	3.5052s

This establishes a $15.30\times$ contraction-interval effect and a $12.8\times$ observed RSS effect for this contribution. It is not a whole-generation speedup. The newly instrumented owner captures are byte-identical to all four original C1/C3 full pre-network inputs, so the ownership diagnostics did not perturb the measured expressions.

GL1: branch copies and nested tensor sums

The actual proper-UV source $140/\{140\}$ family has 18 Taylor-entry branches. All 18 complete mapped numerators are exact substitutions of one 3,780-byte parametric numerator. Their effective edge maps on edges 2–6 are diagonal $\pm$ on-shell energies; the source map is absent and loop carriers are edges 3/5. Every coefficient and denominator is retained in the certificate.

On the supported sign rows, the two organizations are

```
N(sigma2,...,sigma6) * sum_k S_k(sigma)*C_k
sum_k S_k(sigma)*C_k*N(s_k2,...,s_k6).
```

The 32-row selector table contains 18 singly selected rows and 14 unsupported rows. The full factor-preserving diagnostic family shrinks $73,536 \rightarrow 11,602$ text bytes. This proves the early duplication at Taylor entry. It does not claim that the complete post-Taylor coefficients can be replaced with the undifferentiated template.

The owning code is `DirectResidueKey::map_numerator` and the direct Taylor kernel.

`DirectResidueBranches::multiply_key_mapped` applies each complete key before `materialize` combines selector-weighted branches. Source ownership must remain part of that key: of 216 captured GL1 numerator maps, 180 are production maps and 36 are source maps with different retained edge subsets. Equal host IDs alone do not authorize sharing.

The two hottest C3 GL1 contributions account for about 98.9% of tensor execution. They are already gamma-free, so compact slashes cannot explain their contraction cost. Their tensor-valued Add counts are 2,935 and 2,850, versus 191 in C1's hot contribution. Rank-six sums and nested scalar fragmentation are the costly structure; outer rank alone is insufficient.

Recursive extraction of only complete factors shared by every Add arm gives exact within-C3 controls. Function applications, powers and inverses remain opaque. Independent whole-tree proofs certify 821 and 1,638 local extractions, with no changes on a second pass.

Complete input	Original contraction interval	Factored interval	Original/factored peak RSS	Original/factored packed scalar bytes
GL1 term 015	10.655997s	4.319783s	7.0182/2.5890 GB	42,236,243/37,844,104
GL1 term 016	49.460072s	20.148276s	16.6282/8.3263 GB	313,749,436/378,306,347

These are approximately $2.46\times$ improvements with the same executor. Tensor Add counts fall to 1,575/1,897 and rank-six Add counts $40 \rightarrow 4$ and $89 \rightarrow 41$. The complete scalar outputs additionally agree at all three fixed 256-bit points under the unchanged 2^{-128} threshold. This numerical check supplements exact input identities; it is not an exact output or physical phase-space certificate. The attachment metric pair passes the same three-point check. Printed decimal coefficients are converted to independently certified exact integer/dyadic notation before parsing; neither the oracle nor tolerance changes.

Faster contraction does not automatically give a smaller scalar result. Term 016 grows 20.6% in packed output. Its Q-component occurrences rise 8,357,369 $\rightarrow$ 10,194,881; those additional occurrences explain 90.6% of the 139,984,492 additional text bytes. Couplings and `tree_denoms` become globally shared while kinematic expressions repeat more often. Output writing grows 28.254 $\rightarrow$ 33.814 seconds. Evaluator construction and compilation of these factored alternatives have not been benchmarked.

Why the sparse order exceeds 30 GB

The complete problematic attachment term 007 is owned by forest $\{11vs\} \cdot \{168C\} \cdot \{16C4\}$. Its matched C1 contribution keeps twelve gamma factors outside all sums, three outer vector factors, and a rank-three varying sum. C3 has 108 gamma occurrences inside sums: nine copies of its twelve distinct gamma factors, none shared at the outermost product. Nine distinct gamma texts match C1 exactly; three differ only at Lorentz slots identified by C1 metric simplification. This is a matched-owner structural comparison, not a complete cross-version scalar identity.

Both inputs have zero compact gamma arguments. C1 has no rank-five or rank-six tensor sum; C3 exposes nested rank-five and rank-six sums before closure against gamma factors. The relevant component spaces are 64 versus 4096 per rank-three/rank-six tensor term, before accounting for the symbolic size of each component.

On the identical original input, `MinIntermediateCost` completes in 8.31 seconds of watched process time with 1.58 GB peak RSS. The native sparse order (`MinResultRank`) exceeds 30 GB after 20.76 seconds. Before the capped product, two 4096-component tensors already contain about 4.054 GB of packed atoms; a neighbouring rank-five sum contains another 1.080 GB. The final rank-six $\times$ rank-five pairing shares three axes and distributes over a lazy tensor sum.

An exact structural intervention extracts three common vectors from the first two-arm sum and two from its neighbour. Both inner ranks become three. The change survives parsing: the observed sum payloads drop from 8,194/2,050 logical entries to 130 each, about 63.3/67.5 MB. The original capped parent now completes.

The sparse run nevertheless reaches another unchanged, enclosing three-arm rank-six sum. Outer products rebuild its rank-six tensors before closure against the gamma factors. Four lazy terms then contain 16,384 component entries and about 7.635 GB of packed atoms; the following contraction again exceeds 30 GB. The factored sparse run is therefore a measured null result for eliminating the cap, not a failed input-identity proof or evidence that parsing undid the factorization. Lost sharing occurs at multiple nested boundaries.

Extracting eight common gamma factors in a separate contribution is also a null result: the network parse/prepare/contract/resolve interval changes 3.320 $\rightarrow$ 3.237 seconds and process wall 16.944 $\rightarrow$ 17.247 seconds. That extraction does not lower the expensive sum rank. It must not be conflated with the rank-reducing metric experiment.

C6: an exact zero traps relative epsilon expansion

The native phase trace completes all ten projected-numerator tensor simplifications before entering `Vakint`. The captured batch evaluates term 0 in 122 ms and stalls on term 1, inside `ScalarTerms::backend_kernel  $\rightarrow$  epsilon_pole_order  $\rightarrow$  epsilon_series`. Neither tensor execution, zero-certification traversal nor an external integration backend owns this wait.

The complete 565-packed-byte coefficient is an unchanged product of color tensors and a metric multiplied by this scalar factor. Write $A=8i*GC_{10}*GC_{11}^3$ and $D=-4+2*\epsilon$:

$$A + 4*A/D + (-4*A-A*D)/D = 0.$$

The proof retains every spectator factor. The denominator is nonzero at $\epsilon=0$. `Vakint`'s scratch zero check leaves this form intact: its real-positive-power criterion rejects the complex coefficient/inverse, while `collect_by_coefficient()` does not resolve this cancellation.

The locked `Symbolica` relative-series implementation disables statistical zero checks and repeatedly doubles its depth until it finds a nonzero leading term. When the computed coefficients cancel, the series has relative order zero. An algebraically zero input still containing ϵ can therefore keep that loop running. Literal zero takes a different constant path. The implementation is byte-identical in main and C6; C6 newly introduces this caller/input combination.

A separate small reproducer alpha-renames only the captured scalar symbols consistently. Both the full captured scalar factor and the real minimal zero $1+4/D-2*\epsilon/D$ hit their 10-second relative-order limit. Literal zero returns immediately; absolute-order-zero expansion of the captured factor returns zero in 0.477 ms. One

`collect_factors()` call still leaves an unsimplified cancellation and also times out. The larger complete-coefficient capture independently retains the exact native namespaces and attributes.

Pole budgeting needs all negative Laurent terms; it does not need to search for the first nonzero positive-order term. Changing only `epsilon_pole_order` from relative order one to absolute order zero completes the entire captured ten-term batch in 0.833756 seconds of evaluation, 0.913508 seconds total program time. All 47 coefficient pole queries, 57 epsilon-series calls and ten backend/final-series evaluations finish. The original relative-depth batch retains its 60-second timeout on term 1. Native inputs, settings and locked dependencies are unchanged; format, check, build, Clippy and source-integrity audits pass.

This isolates the wait mechanism. The standalone replay has different global Symbolica allocation history from full GammaLoop, which was not rerun with this diagnostic change. Completion does not independently certify all integrated physics values. No production fix is applied.

C7: earlier normalization and parser exposure

C7 adds early `color_simplify()` calls before direct Taylor and cograph numerator mapping. That existing API also simplifies metrics, changing tensor connectivity before Taylor differentiation. Spenso, Idenso, the parser/evaluator implementation and their locked dependencies are unchanged at this boundary.

The complete native attachment input changes from 18 to 15 top-level terms with almost unchanged text size. In the complete candidate pairing, two C6 contributions are compared with a differently grouped C7 contribution containing a new 450,653-byte rank-four sum. An exact C7 owner join is not established for that pair. The complete candidate pair shares 24 of 25 normalized outer factors, including all twelve gamma factors, selector, couplings and denominators. Its remaining tensor-valued Add has two versus five arms. Exact equality of those full residuals has not been established by the bounded normalization check.

Removing only the two new direct-route color calls in an isolated native C7 build restores the complete C6 pre-network expression byte for byte: 1,986,795 UTF-8 bytes, SHA-256

04552e62b442d245f85c8df798b3b99ca122aa3a6f5293babe5381def1d6e0b8. Only outer whitespace is stripped; no signs, indices, namespaces or algebra are normalized. The branch combiner, projected route and local4D changes remain native C7. Source audits cover all 1,892 tracked files before and after the capture; format, check, build and Clippy pass. The capture exits at the explicit pre-parser stop, with all required records present and no tensor-execution stage reached.

This two-call intervention establishes that early color/metric simplification is sufficient to produce the C6→C7 representation change on this workload. It does not establish which call is individually necessary. The native timings illustrate why the downstream scalar matters: C6→C7 tensor work falls 18.724 → 14.730 seconds, while Symbolica construction rises 35.152 → 49.465 seconds and the packed scalar grows 334,489,179 → 394,421,717 bytes. The intervention restores the input to the unchanged backend; it is not a new end-to-end performance measurement.

The immediate C7/C8 GL1 integrated error is already independently reproduced: symmetric color trace `trace(sym(t(a), t(b), t(c)))` incorrectly exposes no adjoint indices. The ordered trace and small projector-expanded sum retain three indices and return the independent signed 6i closure; the native symmetric-plus-antisymmetric sum is rejected as scalar plus tensor. This is an existing public parser limitation exposed by an earlier color reduction, before Vakint integration kernels run.

Validation scope

All timings above are individual guarded observations, with no automatic retries or snapshot updates. One heavy job runs at a time, using eight Rayon workers, four build workers, 128 MiB Rust worker stacks and a 30-decimal-GB process-tree RSS cap. The host is shared. Source files/modes, inputs, recipes and preserved executables are hash-pinned. Each new Rust probe is formatted, checked, built and linted before execution. Complete input proofs use factor-preserving algebra and explicit owner/selector correspondence.

The exact source pins and primary repeats remain in the native measurement report. The controlled factor replays do not certify total generation speed, integrated values, or every contraction strategy. Manual PySecDec integration is excluded. No production code, existing test or published stack commit changes in this investigation. The structural causes and separate failure mechanisms above are established for the recorded workloads; production remedies and their full-pipeline correctness/performance validation remain implementation work.

Evidence and reproduction

The [causal result index](#) identifies the decisive observations and hashes their native receipts. These are diagnostic groups, not repository test-suite totals:

Group	Fresh result
Complete factor replays	Six completed processes: original/factored pairs for attachment 006 and GL1 015/016
Complete scalar-output supplements	Three agreeing pairs, each at three fixed 256-bit points
Sparse controls	Two completed fixed-order runs and two 30 GB sparse-order caps
Minimal epsilon-series matrix	Two completed cases and three bounded relative-series timeouts
C6 depth-only batch	Ten terms, 47 pole queries and 57 epsilon-series calls completed
C7 upstream intervention	One complete input restored exactly; intentional pre-parser stop

The [text evidence archive](#) and [per-file manifest](#) preserve exact inputs, factor proofs, complete multiline traces, source patches, native dependency pins, build checks and command receipts. The native-matrix [companion archive](#) remains required for older source/capture provenance. The new archive stores current investigation artifacts under `files/new/` and selected older prerequisites under `files/prior/`. Within `files/new/`, `causal-results.json` is the entry point. The metric proof is under `hot-structure/attachment6-common-metrics`; GL1 whole-tree proofs are under `hot-structure/gll-recursive-common-factors-04`; the branch certificate is under `hot-structure/gll-proper-uv-sign-template-01`. The sparse owner comparison and complete execution traces are under `attachment7-sparse-profile`. The native zero coefficient is under `c6-coefficient-boundary/complete-coefficient`; the minimal and complete-batch controls are under `c6-series-zero-reproducer` and `c6-absolute-pole-order`. The C7 intervention is under `uv-boundaries/c7-no-early-color`. Every measured command retains its exact argv and working directory in the linked receipt. Rust probes use their pinned Cargo manifest and lockfile with `cargo fmt --check`, `cargo check --locked`, `cargo build --locked` and `cargo clippy --locked`, with the recorded profile/features and wrapper settings. Reproduction requires rebuilding from those pinned sources, a separately configured license and fresh attempt directories; runtime argv must refer to the newly built audited executable. The archive excludes the licensed launcher, process environments, native binaries and full source trees. Large derived scalar outputs are represented by their native hashes, lengths, exact replay inputs and regeneration scripts; the archive does not contain their complete bytes. Setup failures, bounded inconclusive checks and null controls retain their original classifications.